# DATA SCIENCE WITH PYTHON

## INTEREST BASED LOCATION RECOMMENDATION SYSTEM

T A S K - 3

FINAL PROJECT

**Prepared by**
MIHIRKUMAR SUTARIYA

23/01/2026

## Table of Contents


**1. Introduction ..... 2**

**2. Overall System Architecture ..... 2**

**3. Issues Encountered and Resolutions ..... 3**

**4. Data Processing and Manipulation Module ..... 4**

**5. Use Cases ..... 4**

Use Case 1: Recommending Venues for a User & Category ..... 4

Use Case 2: Finding Most Similar Users ..... 5

Use Case 3: Determining Closest Venue for a Group of 5 person ..... 6

**6. Results ..... 7**

Use Case 1: Recommend a list of 10 unvisited locations for any userID and CatagoryID ..... 7

Use Case 2: Identifying the Top 10 Most Similar Users ..... 7

Use Case 3: Determining a Central Meeting Point for Multiple Users ..... 8

**7. Bibliography ..... 8**


## Table of Figures


Figure 1 Output of the 10 unvisited venues ..... 7

Figure 2 Output of the 10 Most Similar Users Identified for User Id 844 using KNN ..... 8

*Figure 3 Result of the nearest-point suggestion for each of the five users* ..... 8

Figure 4 Result of the calculated central point in relation to the individual user locations. ..... 8

# 1. Introduction

The Interest-Based Location Recommendation System is designed to provide intelligent, personalized and meaningful recommendations. It analyses user check-in history, preferences, and interests to identify similar users, suggest relevant venues, and recommend optimal meeting locations for groups. The system combines collaborative filtering, semantic category analysis, and geospatial techniques to ensure recommendations are meaningful, relevant, and convenient. Machine learning algorithms such as K-Nearest Neighbors (KNN), TF-IDF, and cosine similarity are used alongside spatial optimization methods for accurate predictions. The solution has been tested using the FourSquare dataset to validate its performance.

**Key Features:**

- Analyze historical check-ins to understand individual user preferences
- Compute user similarity to provide collaborative recommendations
- Recommend central meeting points for a group of users using geospatial proximity
- Suggest venues to individual users based on category relevance and interests
- Utilize machine learning techniques (KNN, TF-IDF, cosine similarity) for recommendation accuracy
- Implemented entirely in Python using industry-standard data science and machine learning libraries
- Tested with the FourSquare dataset to ensure reliable and meaningful recommendations

# 2. Overall System Architecture

| Layer | Description / Tasks |
|---|---|
| **Data Ingestion Layer** | • Loads raw FourSquare check-in data<br>• Loads cleaned category metadata (from Task 2)<br>• Validates:<br>  ○ Column structure<br>  ○ Data types<br>  ○ Missing or invalid values |
| **Data Preprocessing Layer** | • Removes duplicate check-ins and invalid entries<br>• Enriches dataset with:<br>  ○ Semantic category labels<br>  ○ Hierarchical category paths<br>• Produces a clean dataset suitable for analysis |
| **Feature Engineering Layer** | • Constructs user–location matrix (visits per user per venue)<br>• Converts dense matrices into CSR sparse format for memory efficiency<br>• Prepares additional structures: |

| Layer | Description / Tasks |
|---|---|
| | ○ Visit frequency tables<br>○ Category embeddings for similarity computation |
| **Modeling Layer** | • Computes user similarity using:<br>○ Cosine similarity on user–location vectors<br>○ K-Nearest Neighbors (KNN) for top similar users<br>• Computes category similarity using TF-IDF and cosine similarity |
| **Recommendation Layer** | • Generates personalized venue recommendations for individual users<br>• Computes optimal meeting locations for user groups (geographic centroid and proximity)<br>• Combines user similarity and category similarity for collaborative recommendations |

## 3. Issues Encountered and Resolutions

During the development of the Interest-Based Location Recommendation System, several challenges were encountered. These issues were addressed to ensure accurate, efficient, and meaningful recommendations:

**1. Duplicate and Inconsistent Category Labels:**
The original dataset contained duplicate and inconsistently named venue categories, which could affect recommendation accuracy.
**Resolution:** Cleaned and standardized category labels, added descriptive names, and structured venues into high-level categories with hierarchical paths for semantic clarity.

**2. Sparse and Large User–Venue Matrices:**
The user–venue matrix was high-dimensional and sparse, resulting in high memory usage and slow similarity computations.
**Resolution:** Converted matrices to Compressed Sparse Row (CSR) format to improve memory efficiency and speed for KNN and cosine similarity calculations.

**3. Missing or Invalid Data:**
Some check-ins had duplicate (user IDs, venue IDs, coordinates), which could impact modelling and recommendations.
**Resolution:** Performed data validation during ingestion and ensured all key columns had proper data types and values.

**4. Ensuring Novel Recommendations:**
Users could otherwise be recommended venues they had already visited, reducing relevance.
**Resolution:** Implemented filtering to exclude previously visited venues when generating recommendations, ensuring personalized and novel suggestions.

5. **Optimizing Group Venue Selection:**
Determining a venue closest to multiple users required accurate distance computation and optimization, which could be computationally intensive.
**Resolution:** Used geographic centroid calculations and optimization (scipy.optimize.minimize with geopy.distance) to efficiently find the venue minimizing total travel distance for the group.

6. **Validating Similar User Calculations:**
Sparse interactions and overlapping visits could lead to inaccurate KNN results.
**Resolution:** Validated using overlap counts of visited venues and tuned KNN parameters to ensure the top similar users were correctly identified.

These resolutions ensured that the system produced **accurate, reliable, and meaningful recommendations** for both individual users and groups, while maintaining computational efficiency and robustness.

## 4. Data Processing and Manipulation Module

**Data Loading:** Imported the NYC check-ins dataset (dataset_NYC.txt) and the cleaned category metadata (CategoryNameAndLabels.xlsx), verifying structure, data types, missing values, and indices.

**Duplicate Removal:** Identified and removed duplicate Venue Category IDs and Names, resulting in unique, human-readable category labels.

**Category Enrichment:** Added descriptive category names to ensure clarity and consistency for recommendations.

**Category Structuring:** Organized venues into high-level categories and assigned hierarchical category paths for semantic structure.

**Merging and Final Dataset:** Combined cleaned category information with check-in data to produce a structured dataset ready for analysis and recommendation.

## 5. Use Cases

### Use Case 1: Recommending Venues for a User & Category

**Description:**

- Generate personalized venue recommendations for a user based on a specific category.

- Combine the user's historical preferences with semantic similarity between categories to improve recommendation relevance.
- Ensure that recommendations exclude venues the user has already visited.

**Methodology:**
- Compute category similarity using **TF-IDF** vectorization of category labels and cosine similarity.
- Identify similar users using past check-in behavior (collaborative filtering).
- Aggregate preference scores from similar users and category similarity to rank potential venues.
- Filter out venues already visited by the target user to ensure novelty.

**Tools:**
- **TfidfVectorizer** from **sklearn** for category similarity
- Pandas and NumPy for data manipulation, filtering, and ranking
- Python data structures for efficient aggregation and computation

**Functions:**
- **top_5_similar_categories_tfidf(dataset, categoryID)** → computes top 5 similar categories
- **unvisted_locations(df,UserID)** → identifies venues the user has not visited
- **recommend_venue(checkIn_df, category_df)** → generates top 10 recommendations combining category and user similarity

**Expected Results:**
- Table or DataFrame listing 10 recommended venues
- Each entry includes venue name, Venue CategoryID, and Venue Category Name
- Recommendations are personalized, relevant, and actionable

## Use Case 2: Finding Most Similar Users

**Description:**
- Identify the top 10 users whose interests align most closely with a given user.
- This enables collaborative recommendations by leveraging the preferences of similar users.
- Useful for understanding user behavior patterns and community similarities.

**Methodology:**
- Build a **user–venue interest matrix**: rows represent users, columns represent venues, and values indicate visit frequency.
- Apply **K-Nearest Neighbors (KNN)** algorithm with cosine similarity to compute nearest neighbors in high-dimensional space.
- Optionally validate results by checking overlap in visited venues between users.

**Tools:**
- **scipy.sparse** for efficient storage of large, sparse matrices
- **sklearn.neighbors.NearestNeighbors** for KNN computation
- Python and Pandas for matrix creation and preprocessing

**Functions:**
- **find10mostSimUsers(df)** → takes the user–venue matrix as input and returns the top 10 similar users
- Logic: vectorize users → compute cosine similarity → rank users → return top matches

**Expected Results:**
- DataFrame showing top 10 most similar users for the target user
- Each row includes UserID and similarity score
- Enables downstream collaborative recommendations and analysis of user communities

## Use Case 3: Determining Closest Venue for a Group of 5 person

**Description:**
- Recommend a venue that minimizes travel distance for all users in a group.
- Provides a practical solution for group meetups by combining individual location preferences.
- Ensures fairness by optimizing for geographic centrality rather than individual convenience.

**Methodology:**
- Determine each user's current location based on their most frequently visited venues.
- Compute the **geographic centroid** of all user locations (mean latitude and longitude).
- Identify the venue nearest to this centroid using distance calculations.
- Optimize the selection to minimize the total distance all users must travel.

**Tools:**
- **scipy.optimize.minimize** for optimizing sum of distances
- **geopy.distance** for accurate geographic distance computations
- Pandas and NumPy for preprocessing user and venue location data

**Functions:**
- **most_visited_venue_per_user (df)** → finds the most visited venue per user
- **current_location_users (df)** → determines user location coordinates
- **sum_of_distances(x, points)** → calculates total distance from a candidate point to all users
- **centralPoint(df)** → computes geographic centroid of the group
- **find_nearest_venue (df, reference_point**) → selects the closest venue to the centroid

**Expected Results:**

- Recommended venue near the group's geographic center
- Map visualization showing user locations, centroid, and selected venue
- Minimizes total travel distance while maintaining fairness and convenience

# 6. Results

## Use Case 1: Recommend a list of 10 unvisited locations for any userID and CatagoryID

For a given UserID and CategoryID, the system recommends 10 unvisited venues belonging to semantically similar categories and tailored to the user's interests.

Question/User case 1: Recommend a list of 10 unvisited locations for any userID and CatagoryID

```
result_list = recommend_venue(checkIn, category)
print(result_list)
```

[136] ✓ 2.7s

```
Top 10 Recommended Locations for User 12:
                Venue_ID       Venue_Category_ID \
138997  4c66ed9b19f3c9b6ed5ca1ff  4bf58dd8d48988d1e7941735
185297  4e64846b18a8ce02fdc01c5e  4d4b7105d754a06377d81259
153659  4cf4ba5694feb1f78e7629ba  4bf58dd8d48988d1e7941735
139678  4c6c528423c1a1cd563f19cf  4bf58dd8d48988d1e7941735
126407  4c0c04c5009a0f473200ecbf  4bf58dd8d48988d1e7941735
60741   4abb7c34f964a520d28320e3  4bf58dd8d48988d1e7941735
135529  4c4c27ce959220a1fa749710  4bf58dd8d48988d1e7941735
190067  4e88e7af9a52a2df9baf8fbd  4d4b7105d754a06377d81259
122326  4bf6f66a5ec320a14b1c86d3  4bf58dd8d48988d1e7941735
165574  4d9e38b8009b224bda61e6d1  4bf58dd8d48988d1e7941735

       Venue_Category_Name
138997          Playground
185297  Outdoors & Recreation
153659          Playground
139678          Playground
126407          Playground
60741           Playground
135529          Playground
190067  Outdoors & Recreation
122326          Playground
165574          Playground
```

*Figure 1 Output of the 10 unvisited venues*

## Use Case 2: Identifying the Top 10 Most Similar Users

To demonstrate the functionality, UserID 844 was selected as a sample case. Using a data frame of user check-ins, the system identified the top 10 users with the most similar interests by analyzing historical check-in patterns across all users.

Question/Use case 2: Find the list of top 10 most similar users (i.e. their interests match) in the entire data set for any userID as input.

```
find10mostSimUsers(checkIn)
```

[86] ✓ 2.5s

...

Top 10 similar Users for User 844:

| | ID | Distance (KNN) | user_ranking |
|---|---|---|---|
| 0 | 571 | 0.950134 | 1 |
| 1 | 892 | 0.970882 | 2 |
| 2 | 301 | 0.972783 | 3 |
| 3 | 644 | 0.974998 | 4 |
| 4 | 600 | 0.977700 | 5 |
| 5 | 960 | 0.978240 | 6 |
| 6 | 398 | 0.980069 | 7 |
| 7 | 266 | 0.984355 | 8 |
| 8 | 922 | 0.984578 | 9 |
| 9 | 979 | 0.984610 | 10 |

*Figure 2 Output of the 10 Most Similar Users Identified for User Id 844 using KNN*

## Use Case 3: Determining a Central Meeting Point for Multiple Users

The system calculates the central location for a group of 5 users by analyzing their frequently visited venues, recommending a meeting point close to all participants.

Question/User case 3: Determine closest Venue for a group of users

```
meeting_venue = common_nearest_venue(checkIn)
```

[148] ✓ 36.9s

...

Current location

| | ID | Latitude | Longitude |
|---|---|---|---|
| 0 | 59 | 40.783910 | -73.836108 |
| 1 | 911 | 40.692337 | -74.087248 |
| 2 | 119 | 40.758457 | -73.989140 |
| 3 | 420 | 40.749783 | -74.004073 |
| 4 | 590 | 40.728787 | -73.851986 |

Nearest Point to all users

[ 40.75661721 -73.98873859]

Suggested Meeting Location

| | |
|---|---|
| Venue_ID | 4ab52081f964a520a57220e3 |
| Venue_Category_Name | Hotel |
| Latitude | 40.756648 |
| Longitude | -73.988705 |
| distance | 0.004436 |

Name: 60480, dtype: object

*Figure 3 Result of the nearest-point suggestion for each of the five users*

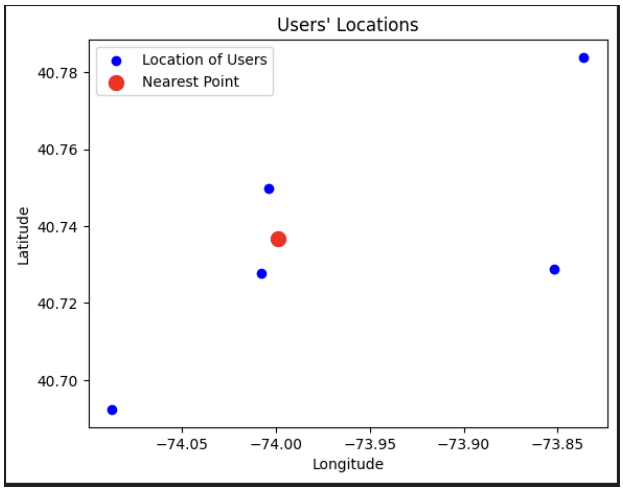



*Figure 4 Result of the calculated central point in relation to the individual user locations.*

# 7. Bibliography

For user similarity recommendations: Su, X., & Khoshgoftaar, T. M. (2009). *A survey of collaborative filtering techniques*. Advances in Artificial Intelligence https://doi.org/10.1155/2009/421425

Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001). *Item-based collaborative filtering recommendation algorithms*. Proceedings of the 10th International Conference on World Wide Web (WWW '01), 285–295. https://doi.org/10.1145/371920.372071

OpenAI. (2023). ChatGPT [GPT-5.1]. https://chat.openai.com

Naik, Krish. "Tutorial 2- Creating Recommendation Systems using Nearest Neighbors" Youtube, www.youtube.com/watch?v=kccT0FVK6OY&list=PLZoTAELRMXVN7QGpcuN-Vg35Hgjp3htvi&index=10

Prof. Mr. Lengyel (2025). Lecture slides